Лабораторная работа №4 «Виртуализация и контейнеризация в DevOps»

Цель: Освоить основы контейнеризации с помощью Docker и Docker Compose, упаковать веб-приложение с ML-моделью в контейнер, организовать его взаимодействие с внешней базой данных PostgreSQL на ВМ1, обеспечить воспроизводимость запуска и подготовить приложение к дальнейшей автоматизации в CI/CD. Заложить основу для многоконтейнерной архитектуры сквозного проекта «ML Prediction Service».

Задачи:

1. Создание снапшота ВМ2 (app-server) перед началом работы.
2. Установка Docker и Docker Compose на ВМ2.
3. Написание Flask-приложения, которое загружает обученную ML-модель и обрабатывает POST-запросы /predict.
4. Создание app/requirements.txt с зависимостями.
5. Написание docker/Dockerfile для сборки образа приложения.
6. Сборка Docker-образа, запуск контейнера, проверка работы эндпоинта.
7. Настройка подключения контейнера к PostgreSQL на ВМ1 (SSL, переменные окружения).
8. Создание docker/docker-compose.yml для управления контейнером.
9. Проверка записи предсказаний в БД с ВМ1.
10. Коммит Dockerfile и docker-compose.yml в Git-репозиторий.

Критерии оценивания (на отчёте к каждому критерию может быть задан вопрос, ответ на который повлияет на итоговый балл):

– Docker и Docker Compose установлены, сервис работает (1 балл);

– Написано Flask-приложение с ML-моделью, корректно обрабатывающее POST-запросы (1 балл);

– Dockerfile корректен, образ успешно собирается (1 балл);

– Контейнер запускается, отвечает на запросы, подключается к PostgreSQL на ВМ1 (1 балл);

– Docker Compose работает, предсказание записано в БД; отчёт с командами и скриншотами (1 балл).

Теоретическая справка

Docker – платформа контейнеризации, позволяющая упаковать приложение со всеми зависимостями в изолированный образ. Контейнеры используют ядро хоста, что делает их легче виртуальных машин и обеспечивает быстрый запуск.

Docker-образ – неизменяемый шаблон, содержащий файловую систему и метаданные. Docker-контейнер – запущенный экземпляр образа. Dockerfile – текстовый файл с инструкциями по сборке образа. Docker Compose – инструмент для описания и запуска многоконтейнерных приложений через YAML-файл.

Ключевые инструкции Dockerfile:

- FROM – базовый образ;
- WORKDIR – рабочая директория внутри контейнера;
- COPY – копирование файлов;
- RUN – выполнение команд при сборке;
- EXPOSE – объявление порта;
- CMD – команда запуска при старте контейнера.

Переменные окружения позволяют передавать параметры подключения к БД без изменения образа. Это важно для безопасности и переносимости.

Пошаговая инструкция

Подготовка и снапшот ВМ2

1. В веб-интерфейсе Proxmox выберите ВМ2 (app-server) – Snapshots – Take Snapshot. Имя: before_docker_lab4.

2. Подключитесь к ВМ2 по SSH.

3. Обновите систему.

Установка Docker и Docker Compose

1. Выполните:

```
sudo apt install docker.io docker-compose -y
sudo systemctl status docker
```

2. Добавьте пользователя student в группу docker, чтобы работать без sudo:

```
sudo usermod -aG docker $USER
newgrp docker
```

3. Проверьте:

```
docker --version
docker-compose --version
docker run hello-world
```

Написание Flask-приложения с ML-моделью

1. Перейдите в репозиторий:

```
cd ~/ml-prediction-service
```

Важно: перед написанием приложения создайте и обучите простую ML-модель (или возьмите готовую модель). Например, можно взять логистическую регрессию на датасете Iris (у Вас должна быть другая модель и

по необходимости другие файлы обучения и flask-приложения). Сохраните её в model/model.pkl.

2. Пример скрипта обучения (сохраните в model/train.py):

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
import pickle
X, y = load_iris(return_X_y=True)
model = LogisticRegression(max_iter=200)
model.fit(X, y)
with open("model/model.pkl", "wb") as f:
    pickle.dump(model, f)
print("Model trained and saved to model/model.pkl")
```

3. Запустите:

```
cd ~/ml-prediction-service
python3 model/train.py
```

Убедитесь, что файл model/model.pkl создан. Не коммитьте его – в .gitignore уже есть правило model/*.pkl.

4. Теперь создайте app/app.py:

```
from flask import Flask, request, jsonify
import psycopg2, json, os, pickle
app = Flask(__name__)
# Загрузка модели при старте
MODEL_PATH = os.getenv("MODEL_PATH", "model/model.pkl")
with open(MODEL_PATH, "rb") as f:
    model = pickle.load(f)
def get_db():
    return psycopg2.connect(
        host=os.getenv("DB_HOST", "192.168.1.101"),
        dbname=os.getenv("DB_NAME", "ml_predictions"),
        user=os.getenv("DB_USER", "ml_app"),
```

```
        password=os.getenv("DB_PASSWORD", "StrongPassword"),
        sslmode=os.getenv("DB_SSLMODE", "require")
    )
@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"})
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json(force=True)
    features = data.get("features", [])
    if not features:
        return jsonify({"error": "No features provided"}), 400
    pred = model.predict([features])[0]
    proba = max(model.predict_proba([features])[0])
    result = {"class": int(pred), "probability": float(proba)}
    # Запись в БД
    conn = get_db()
    cur = conn.cursor()
    cur.execute(
        "INSERT INTO predictions (input_data, prediction, model_version) VALUES
(%s, %s, %s)",
        (json.dumps({"features": features}), json.dumps(result), "v1.0")
    )
    conn.commit()
    cur.close()
    conn.close()
    return jsonify(result)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

5. Обновите app/requirements.txt:

```
cat > app/requirements.txt <<EOF
flask==2.2.3
psycopg2-binary==2.9.6
scikit-learn==1.2.2
EOF
```

Написание Dockerfile

Отредактируйте docker/Dockerfile:

```
FROM python:3.9-slim
WORKDIR /app
# Установка зависимостей
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# Копирование кода приложения и модели
COPY app/ .
COPY model/ /model/
EXPOSE 5000
CMD ["python", "app.py"]
```

Важно: модель model.pkl копируется внутрь образа. В реальном проекте её можно монтировать как volume, но для учебной задачи допустимо встраивание.

Сборка и запуск контейнера вручную

1. Соберите образ:

```
cd ~/ml-prediction-service
docker build -t ml-app:v1 -f docker/Dockerfile .
```

2. Проверьте, что образ создан:

```
docker images | grep ml-app
```

3. Запустите контейнер с переменными окружения для подключения к БД на ВМ1:

```
docker run -d --name ml-app \
  -p 5000:5000 \
  -e DB_HOST=192.168.1.101 \
  -e DB_NAME=ml_predictions \
  -e DB_USER=ml_app \
  -e DB_PASSWORD=StrongPassword \
  -e DB_SSLMODE=require \
  -e MODEL_PATH=/model/model.pkl \
  ml-app:v1
```

4. Проверьте логи:

```
docker logs ml-app
```

5. Проверьте работу эндпоинта:

```
curl http://localhost:5000/health
curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d '{"features": [5.1, 3.5, 1.4, 0.2]}'
```

6. Ожидаемый ответ – JSON с классом и вероятностью, например:

```
{"class": 0, "probability": 0.98}
```

7. Подключитесь к ВМ1:

```
ssh student@192.168.1.101
sudo -u postgres psql -d ml_predictions -c "SELECT * FROM predictions ORDER BY id DESC LIMIT 5;"
```

Вы должны увидеть запись с input_data, prediction и model_version. Если запись есть – интеграция приложения с БД работает.

Создание docker-compose.yml

1. Отредактируйте docker/docker-compose.yml:

```
version: "3.8"
services:
  app:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    image: ml-app:v1
    container_name: ml-app
    ports:
      - "5000:5000"
    environment:
      DB_HOST: 192.168.1.101
      DB_NAME: ml_predictions
      DB_USER: ml_app
      DB_PASSWORD: StrongPassword
      DB_SSLMODE: require
      MODEL_PATH: /model/model.pkl
    restart: unless-stopped
```

2. Остановите и удалите контейнер, запущенный вручную:

```
docker stop ml-app && docker rm ml-app
```

3. Запустите через Compose:

```
cd ~/ml-prediction-service
docker-compose -f docker/docker-compose.yml up -d --build
```

4. Проверьте:

```
docker-compose -f docker/docker-compose.yml ps
curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d '{"features": [6.7, 3.0, 5.2, 2.3]}'
```

5. Выполните docker-compose ps.

6. Добавьте файлы в репозиторий:

```
cd ~/ml-prediction-service
git add app/app.py app/requirements.txt docker/Dockerfile docker/docker-compose.yml model/train.py
git commit -m "Add Docker setup: Flask app with ML model and Docker Compose"
git push origin main
```

Оформление отчёта

Отчёт по лабораторной работе должен содержать:

1. Титульный лист (шапку) с названием работы, ФИО студента, группой и пр.
2. Цель и задачи работы.
3. Используемое оборудование и ПО.
4. Выбранная модель или датасет для обучения модели.
5. Ход работы со скриншотами каждого важного шага.
6. Выводы: что было сделано, какие команды использовались, как обеспечена связь контейнера с БД, почему пароль передаётся через переменные окружения.